Applied Digital Logic Design (ECE101)

Dr. Charishma

Department of E&CE

N.M.A.M. Institute of Technology, Nitte

UNIT - I

Fundamentals of Digital Design:

Difference between Analog and Digital Signals, Number Systems: Decimal, Binary, Octal and Hexadecimal. Binary Addition and Subtraction, Digital Logic Gates, Boolean Algebra, Boolean Functions: Canonical Forms, Completely and Incompletely Specified Functions.

06 Hrs

Difference between Analog and Digital Signals

Differences of Analog and Digital Signals

Analog Signals	Digital Signals
Continuous both in time and amplitude	Discrete in time and quantized in amplitude
Infinite range of values	Finite range of values
More exact values, but more difficult to work with	Not as exact as analog, but easier to work with
Storing such a signal requires large amount of memory	Storing such a signal requires less amount of memory
Processing requires large processing power or more time	Processing requires low processing power or less time
Transmitting requires a large bandwidth	Transmitting requires a less bandwidth than analog

Logic Gates

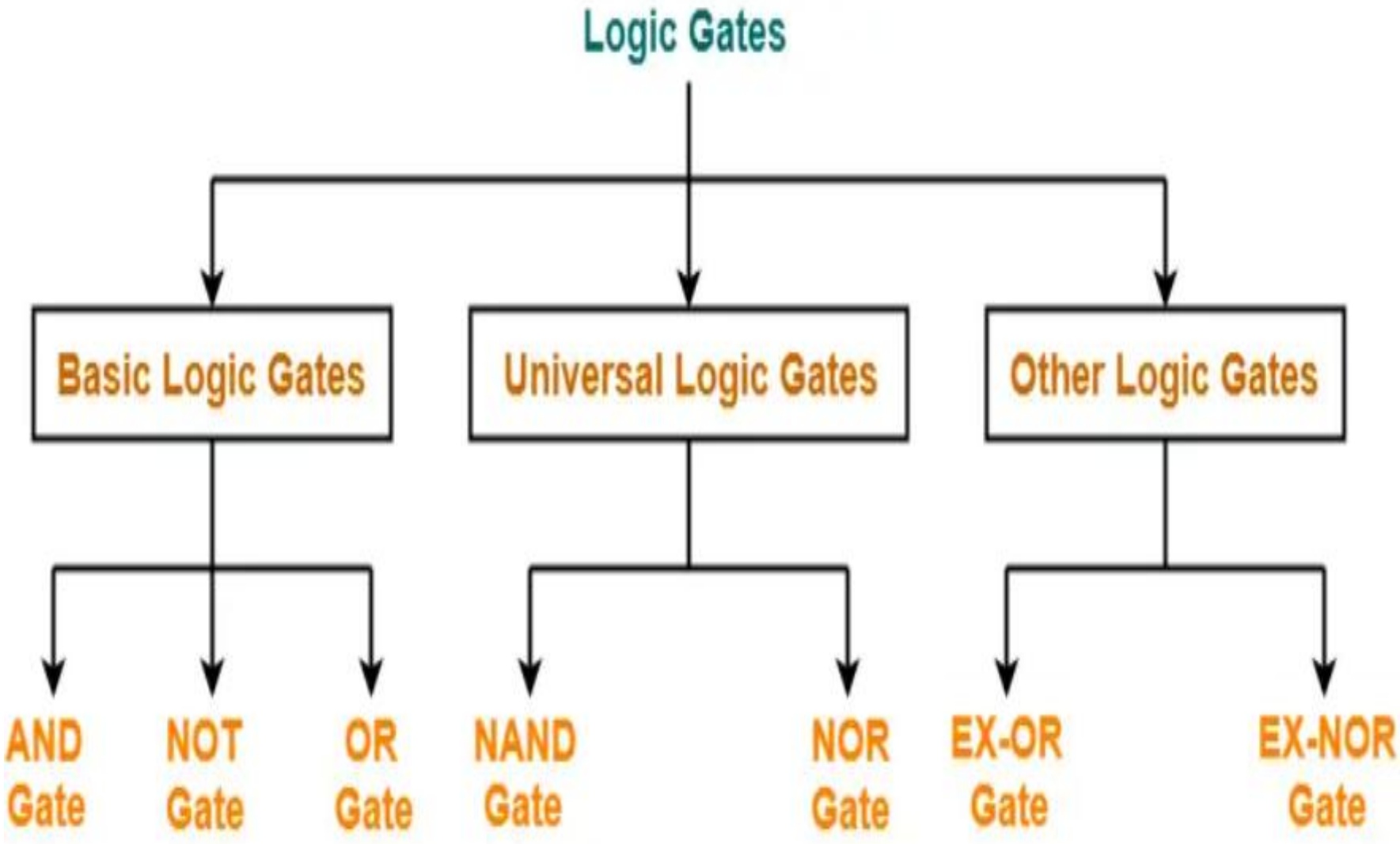
Digital logic circuits handle data encoded in binary form, i.e. signals that have only two values, 0 and 1.

Binary logic dealing with “true” and “false” comes in handy to describe the behavior of these circuits: 0 is usually associated with “false” and 1 with “true.”

A logic gate is a device that acts as a building block for digital circuits . They perform basic logical functions that are fundamental to digital circuits.

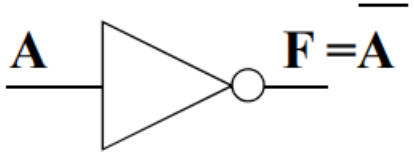
In a circuit, logic gates will make decisions based on a combination of digital signals coming from its inputs .

Logic Gates


Logic Gates
Basic Logic Gates
Universal Logic Gates
Other Logic Gates
AND Gate
NOT Gate
OR Gate
NAND Gate
NOR Gate
EX-OR Gate
EX-NOR Gate

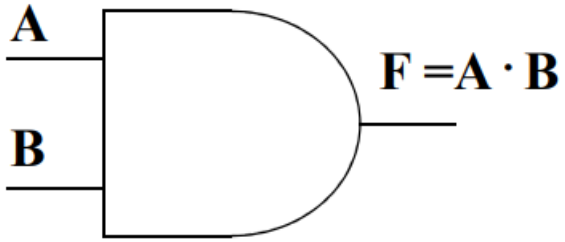
A	$\overline{A}$
0	1
1	0

NOT



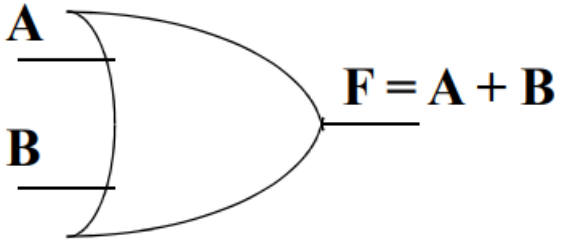
A	B	A · B
0	0	0
0	1	0
1	0	0
1	1	1

AND



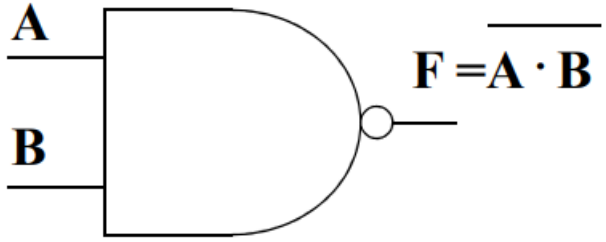
A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	1

OR



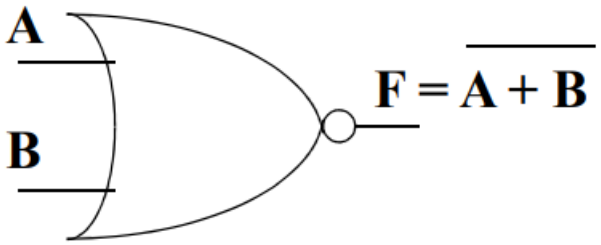
A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

NAND



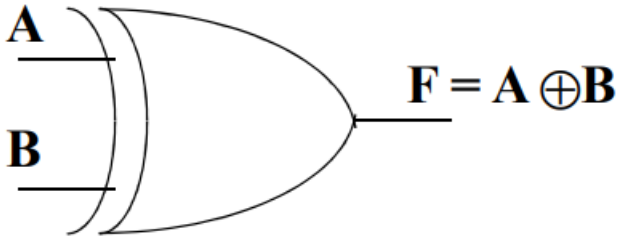
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

NOR



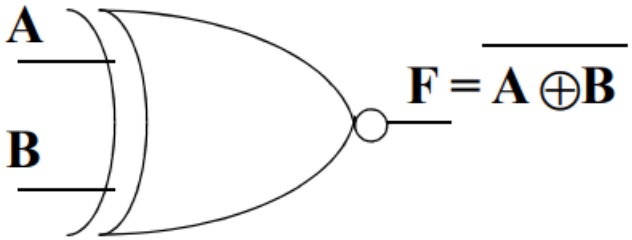
A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

XOR



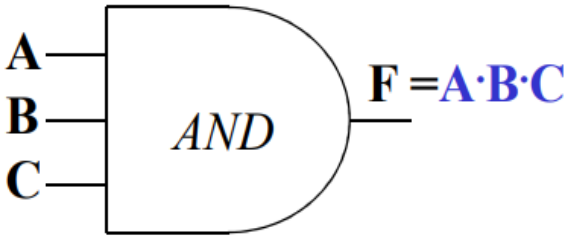
A	B	$\overline{A \oplus B}$
0	0	1
0	1	0
1	0	0
1	1	1

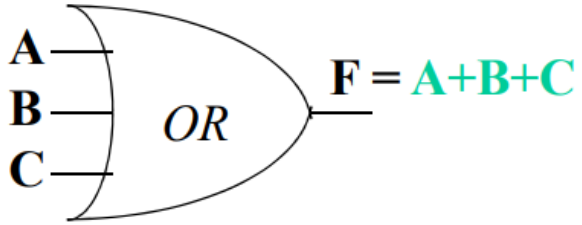
XNOR

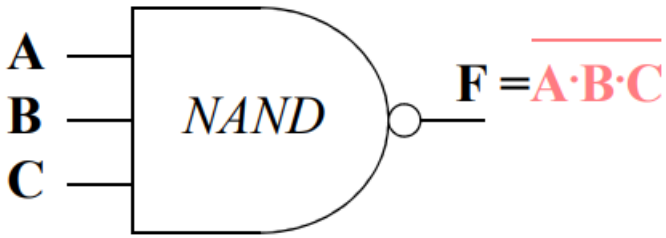


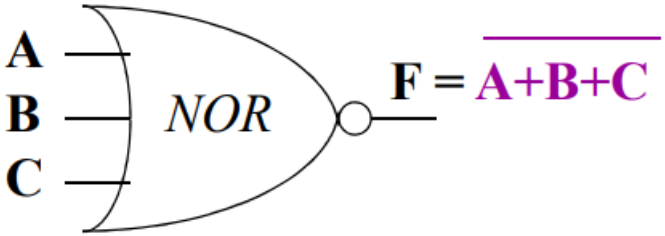
GATES ... with more inputs

A	B	C	A·B·C	A+B+C	$\overline{A \cdot B \cdot C}$	$\overline{A+B+C}$
0	0	0	0	0	1	1
0	0	1	0	1	1	0
0	1	0	0	1	1	0
0	1	1	0	1	1	0
1	0	0	0	1	1	0
1	0	1	0	1	1	0
1	1	0	0	1	1	0
1	1	1	1	1	0	0









Boolean Algebra

Developed by English Mathematician George Boole in between 1815 - 1864.

It is described as an algebra of logic or an algebra of two values i.e True or False.

The term logic means a statement having binary decisions i.e True/Yes or False/No.

It is used to perform the logical operations in digital computer.

In digital computer True represent by '1' (high volt) and False represent by '0' (low volt)

Logical operations are performed by logical operators. The fundamental logical operators are:

1. AND (conjunction)
2. OR (disjunction)
3. NOT (negation/complement)

Commutative Laws

The **commutative laws** are applied to addition and multiplication. For addition, the commutative law states

In terms of the result, the order in which variables are ORed makes no difference.

$$A + B = B + A$$

For multiplication, the commutative law states

In terms of the result, the order in which variables are ANDed makes no difference.

$$AB = BA$$

Associative Laws

The **associative laws** are also applied to addition and multiplication. For addition, the associative law states

When ORing more than two variables, the result is the same regardless of the grouping of the variables.

$$A + (B + C) = (A + B) + C$$

For multiplication, the associative law states

When ANDing more than two variables, the result is the same regardless of the grouping of the variables.

$$A(BC) = (AB)C$$

Distributive Law

The **distributive law** is the factoring law. A common variable can be factored from an expression just as in ordinary algebra. That is

$$A + BC = (A+B)(A+ C)$$

$$A(B+ C) = AB + AC$$

Boolean Algebra

A	B	C	A· (B+C)	A·B+A·C
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

A	B	C	A + B·C	(A+B) · (A+C)
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Rules of Boolean Algebra

1. $A + 0 = A$
2. $A + 1 = 1$
3. $A \cdot 0 = 0$
4. $A \cdot 1 = A$
5. $A + A = A$
6. $A + \overline{A} = 1$
7. $A \cdot A = A$
8. $A \cdot \overline{A} = 0$
9. $\overline{\overline{A}} = A$
10. $A + AB = A$
11. $A + \overline{A}B = A + B$
12. $(A + B)(A + C) = A + BC$

Boolean Algebra

$$
\begin{aligned}
(A + B)(A + C) &= AA + AC + AB + BC \\
&= A + AC + AB + BC \\
&= A(1 + C + B) + BC \\
&= A \cdot 1 + BC \\
&= A + BC
\end{aligned}
$$

DeMorgan's 1st Theorem

The complement of a product of variables is equal to the sum of the complemented variables.

$$\overline{AB} = \overline{A} + \overline{B}$$

Inputs		Output	
A	B	$\overline{AB}$	$\overline{A} + \overline{B}$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

DeMorgan's 2nd Theorem

The complement of a sum of variables is equal to the product of the complemented variables.

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Applying DeMorgan's second theorem to gates:

Inputs		Output	
A	B	$\overline{A + B}$	$\overline{A}\,\overline{B}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

Boolean Expression Simplification

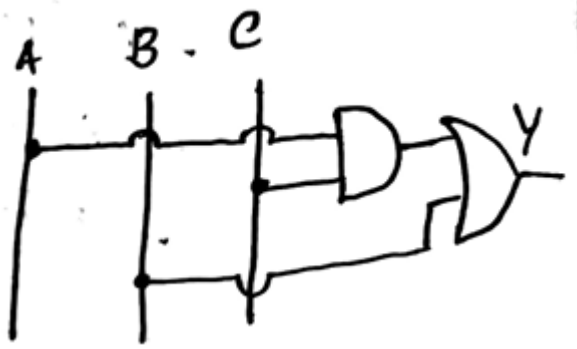
Simplify the given Boolean expression using Boolean algebra and realize using basic gates

i) $Y = AB + \bar{A}(B+C) + B(B+C)$

$= AB + \bar{A}B + AC + BB + BC$

$= AB + AC + B$

$= \underline{B + AC}$



Boolean Expression Simplification

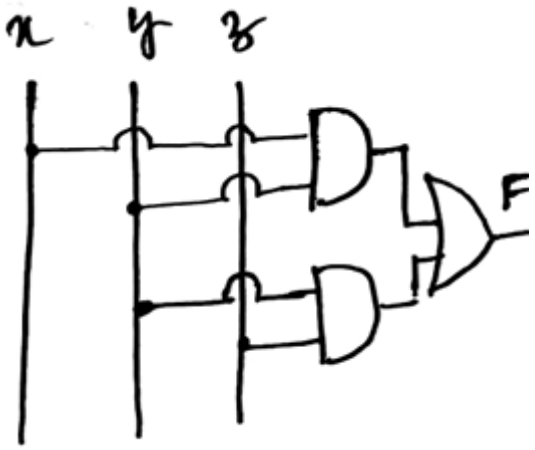
Simplify the given Boolean expression using Boolean algebra and realize using basic gates

ii) $F = xy + xyz + xy\bar{z} + \bar{x}yz$

$= xy + \bar{x}yz$

$= y(x + \bar{x}z)$

$= y(x + z) \quad = xy + yz$



Boolean Expression Simplification

Simplify the given Boolean expression using Boolean algebra and realize using basic gates

iii) $F = \overline{\bar{x}\bar{y} + z} + z + xy + wz$

$= \overline{\bar{x}\bar{y}} \cdot \bar{z} + z + xy + wz$

$= (x+y)\bar{z} + z + xy + wz$

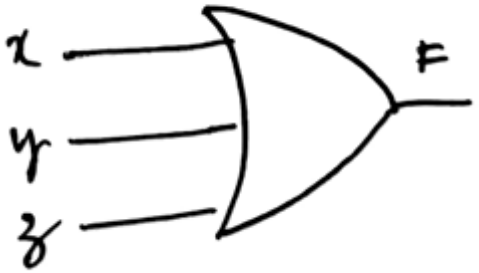
$= x\bar{z} + y\bar{z} + z + xy + wz$

$= x\bar{z} + y\bar{z} + z + xy$

$= x\bar{z} + y + z + xy$

$= x\bar{z} + y + z$

$= x + y + z$



Boolean Expression Simplification

Simplify the given Boolean expression using Boolean algebra and realize using basic gates

(iv) $Y = (A + \bar{B} + \bar{C})(A + \bar{B} + C)$

$= A A + A\bar{B} + AC + \bar{B}A + \bar{B}\bar{B} + \bar{B}C + A\bar{C} + \bar{B}\bar{C} + C\bar{C}$

$= A + A\bar{B} + AC + \bar{B}A + \bar{B} + \bar{B}C + A\bar{C} + \bar{B}\bar{C}$

$= A(1 + \bar{B}) + AC + \bar{B}(A+1) + \bar{B}C + A\bar{C} + \bar{B}\bar{C}$

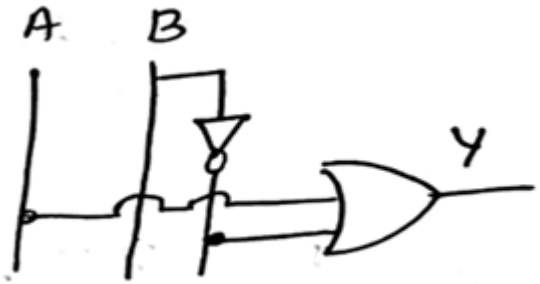
$= A + AC + \bar{B} + \bar{B}C + A\bar{C} + \bar{B}\bar{C}$

$= A(1+C) + \bar{B}(1+C) + A\bar{C} + \bar{B}\bar{C}$

$= A + \bar{B} + A\bar{C} + \bar{B}\bar{C}$

$= A + \bar{B} + A\bar{C}$

$= A + \bar{B}$



Number System

Number is used to refer to amounts of things. Numbers are represented by a string of digits

A number of base, or radix, ‘r’ is a system that uses distinct symbols of r digits.

The four main number systems used in digital systems are

(i) Decimal
(ii) Binary
(iii) Hexadecimal
(iv) Octal

Common Number Systems

System	Base	Symbols	Used by humans?	Used in computers?
Decimal	10(D)	0, 1, … 9	Yes	No
Binary	2(B)	0, 1	No	Yes
Octal	8(O)	0, 1, … 7	No	No
Hexa-decimal	16(H)	0, 1, … 9, A, B, … F	No	No

Table of Number Systems

DECIMAL	BINARY	HEXADECIMAL	OCTAL
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	8	10
9	1001	9	11
10	1010	A	12
11	1011	B	13
12	1100	C	14
13	1101	D	15
14	1110	E	16
15	1111	F	17

Binary Addition and Subtraction

It is possible to add and subtract binary numbers in a similar way to base 10 numbers.

For example, 1 + 1 + 1 = 3 in base 10 becomes 1 + 1 + 1 = 11 in binary.

In the same way, 3 – 1 = 2 in base 10 becomes 11 – 1 = 10 in binary.

Key *Addition* Results for Binary Numbers

$$1 + 0 = 1$$
$$1 + 1 = 10$$
$$1 + 1 + 1 = 11$$

Key *Subtraction* Results for Binary Numbers

$$1 - 0 = 1$$
$$10 - 1 = 1$$
$$11 - 1 = 10$$

Subtraction Using 2's Compliment Method

At first, find 2's complement of the B(subtrahend)

Then add it to the A(minuend)

If the final carryover of the sum is 1, then it is dropped and the result is positive

If there is no carryover, then 2's complement of the sum is the final result and it is negative

Subtraction Using 2's Compliment Method

Find the Subtraction of 110 and 101 using 2's complement method

Here A = 110, B = 101.

Find A - B =? using 2's complement

First find 2's complement of B = 101

2's complement of a number is 1 added to it's 1's complement number

1's complement of 101 is 010

Now add 1 : 010 + 1 = 011

Now Add this 2's complement of B to

	1	1		
		1	1	0
+		0	1	1
	1	0	0	1

The leftmost bit of the result is called carry and it is ignored.

So the answer is 001

Subtraction Using 2's Compliment Method

Find Subtraction of 10110 and 11101 using 2's complement method

Here A = 10110, B = 11101.
Find A - B =? using 2's complement

First find 2's complement of B = 11101

2's complement of a number is 1 added to it's 1's complement number

1's complement of 1's complement of 11101 is 00010

Now add 1 : 00010 + 1 = 00011

Now Add this 2's complement of B to A

```
      1  1
   1  0  1  1  0
+  0  0  0  1  1
-----------------
   1  1  0  0  1
```

Here there is no carry, answer is - (2's complement of the sum obtained 11001)

Subtraction Using 2's Compliment Method

2's complement of a number is 1 added to its 1's complement number

1's complement of 11001 is 00110

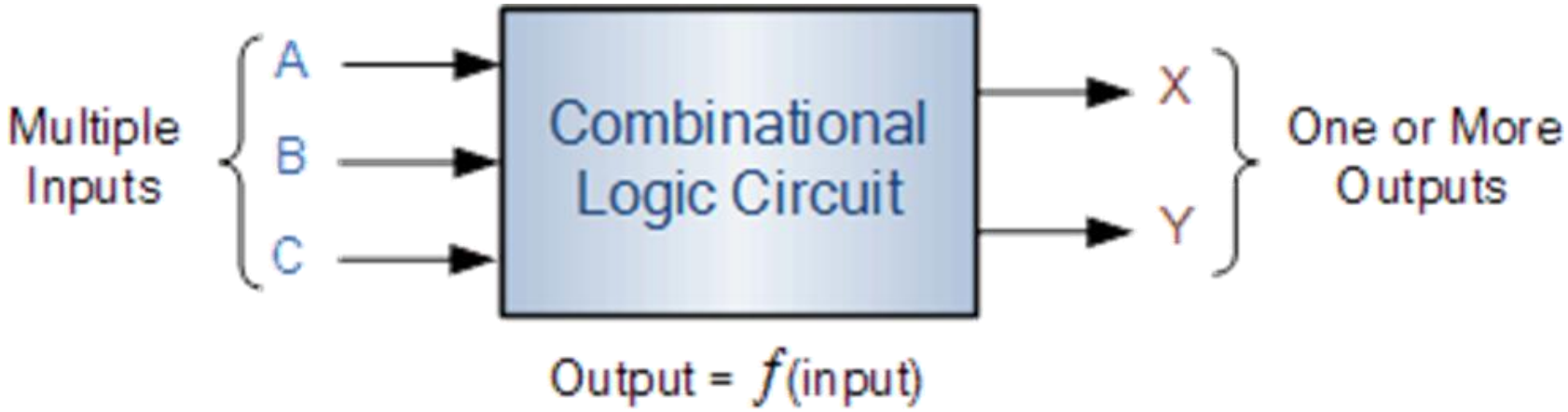
Now add 1 : 00110 + 1 = 00111

So the answer is -00111

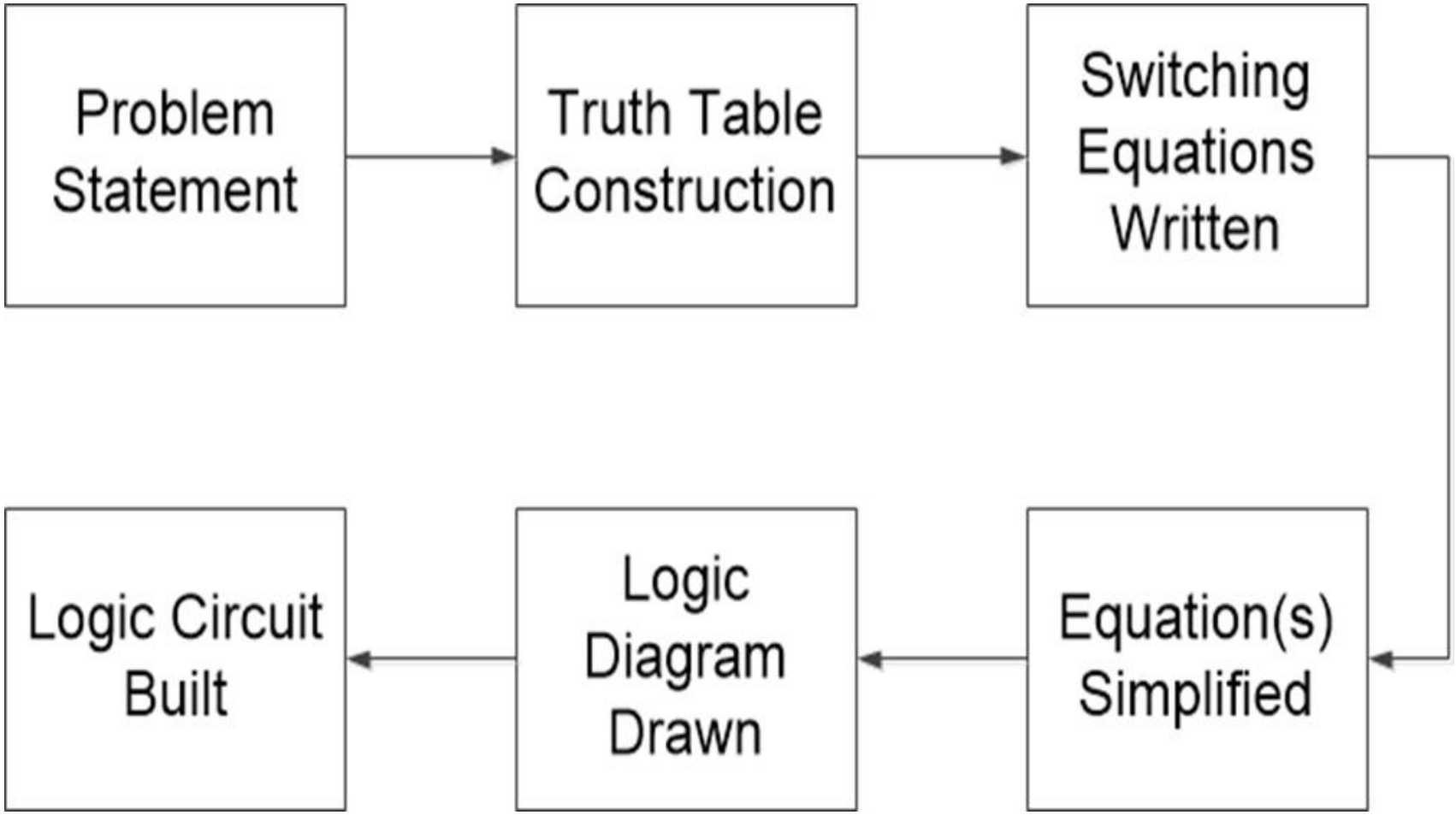
Definition of Combinational Logic

- Combinational Logic deals with the techniques of "combining" the basic gates into circuits that perform some desired function.
- Examples – Adders, Subtractors, Decoders. Encoders, Multipliers etc.
- Logic circuits **without feedback** from output to input.
- Logic circuits that contain **no memory**

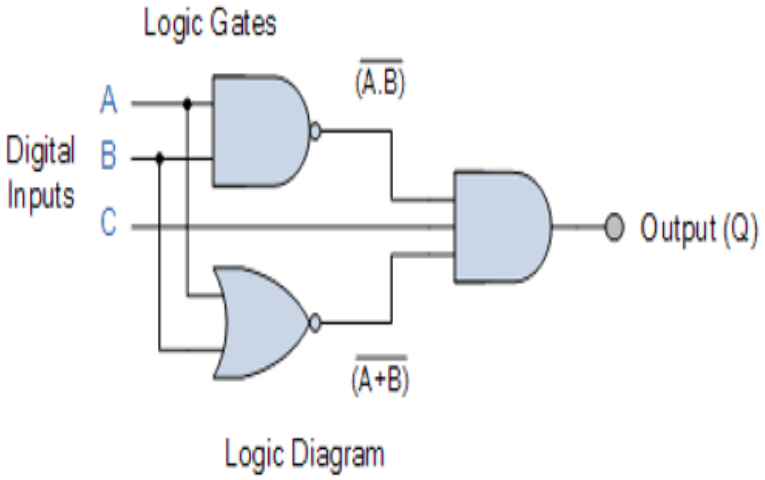
Definition of Combinational Logic



General Logic Design Sequence



- Three ways
  1. **Logic Diagram**: Graphical representation
  2. **Truth Table**: A list that shows all the output states in tabular form for each possible combination of input variable.
  3. **Boolean Equations**: Algebraic expression showing the operation of the logic circuit for each input variable.



Logic Diagram

Typical Truth Table

C	B	A	Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Boolean Expression

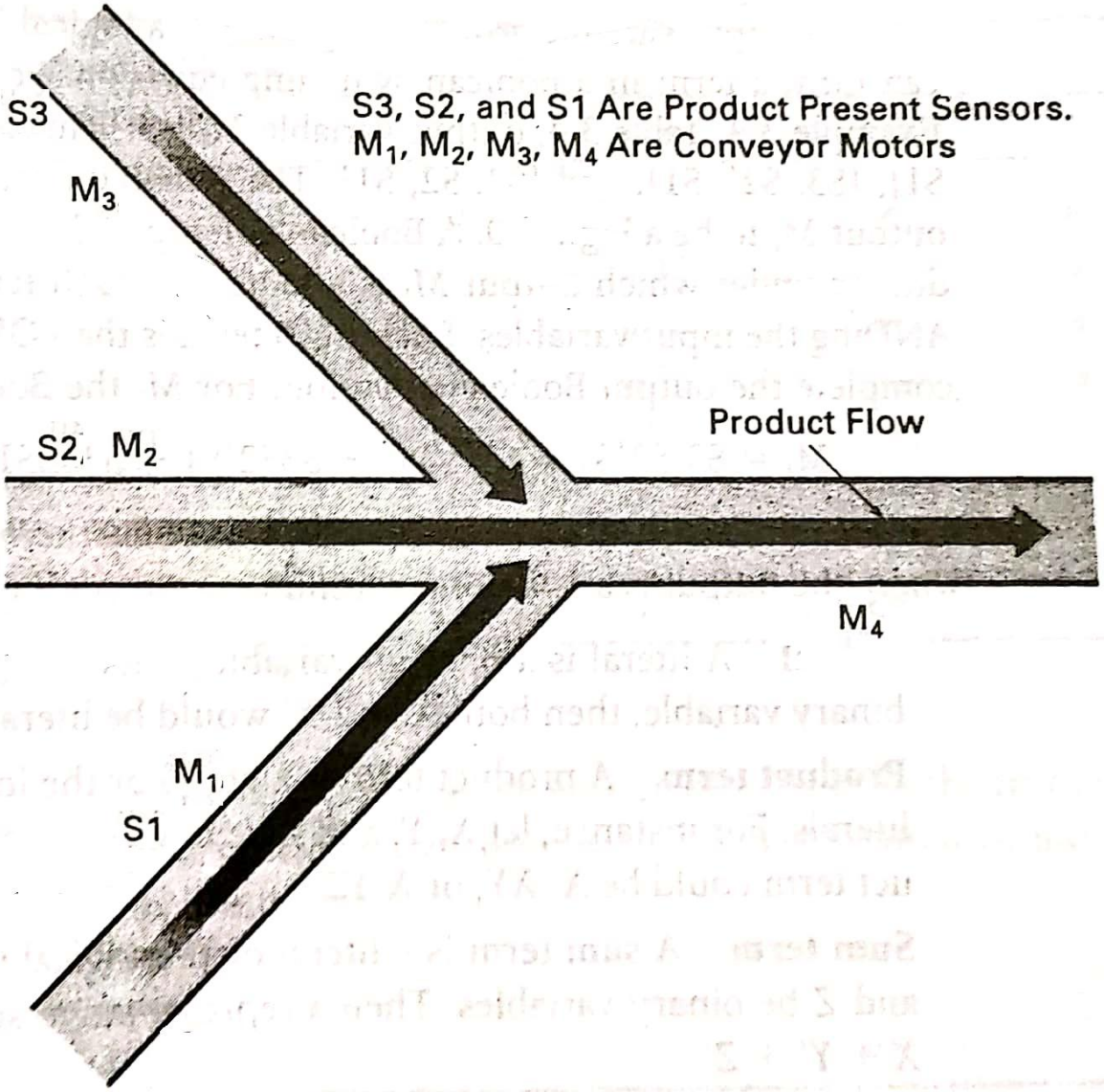
$$Q = \overline{(A.B)}.\overline{(A+B)}.C$$

Problem statements to Truth Table

A conveyor system brings raw materials in from three different sources. The three sources converge into a single output conveyor. Sensors mounted adjacent to each source conveyor indicate the presence of raw material. All four conveyors have separate motors so they can be individually controlled. Each source conveyor can have a different speed. The output product flow rate is fixed; it can be turned only ON or OFF. The output product rate must match the source flow rates. To accomplish this the following conditions must be met. If source 1 has product, then sources 2 and 3 must be turned OFF; if the source 1 is empty, then either 2 or 3 or both can be turned ON. If no product is available from the three sources, the output conveyor must be turned OFF. The respective source conveyor must be turned OFF if no product is available.

Problem statements to Truth Table

Conveyor system



Problem statements to Truth Table

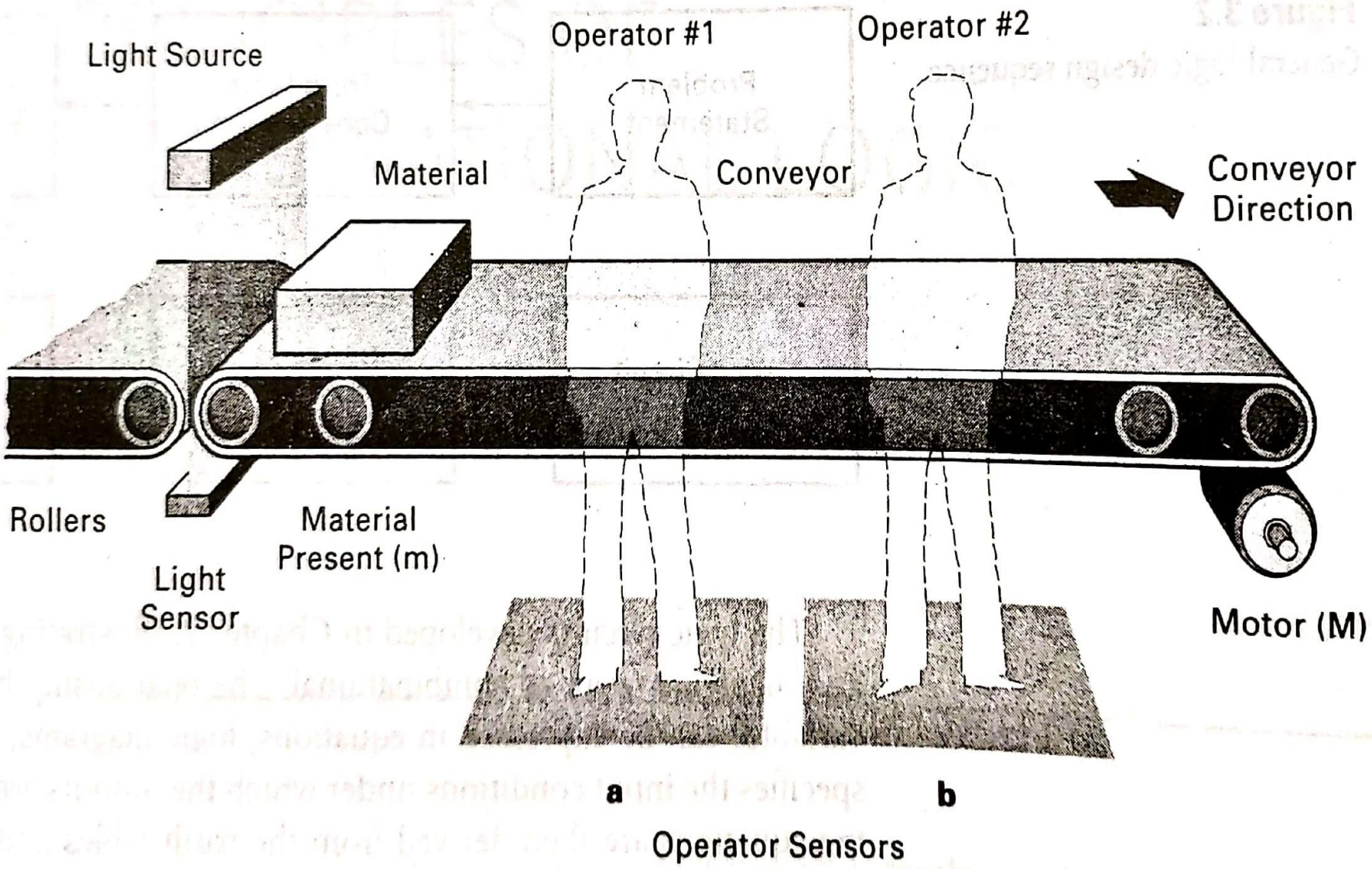
S3	S2	S1	M_4	M_3	M_2	M_1
0	0	0	0	0	0	0
0	0	1	1	0	0	1
0	1	0	1	0	1	0
0	1	1	1	0	0	1
1	0	0	1	1	0	0
1	0	1	1	0	0	1
1	1	0	1	1	1	0
1	1	1	1	0	0	1

Problem statements to Truth Table

An electric motor powering a conveyor used to move material is to be turned on when one of two operators is in position, if material is present to be moved and if the protective interlock switch is not open. Input and output variables are to be expressed in binary; that is, if operator 1 is in position, then the associated variable is a logical 1. The motor is running (on) if its output control variable is a 1, and the motor is off if the variable is a 0. Figure shows a simple diagram of the conveyor system.

The first task is to identify the input and output variables and to assign names to them.

Problem statements to Truth Table



Problem statements to Truth Table

a	b	m	s	M
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Design of Combinational Circuit

Design a circuit for adding two single binary digits (bits) and producing two outputs: the sum of those digits ("S") and a carry bit ("C") that is generated only if both input bits are 1.

(Essentially performing the basic addition operation on the least significant bits of two binary numbers while only considering the potential for a single carry bit.)

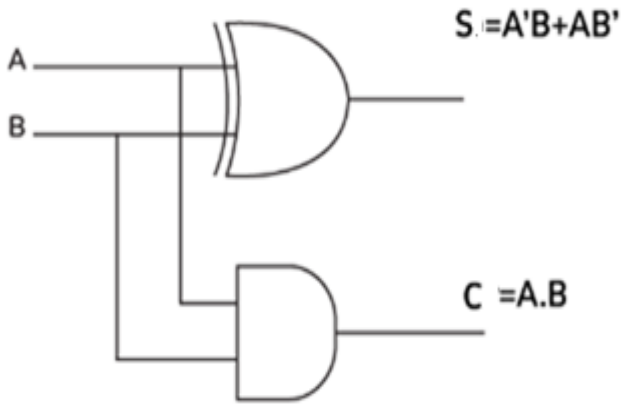
Truth Table

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

S =A'B+AB'

C =A.B

Half Adder



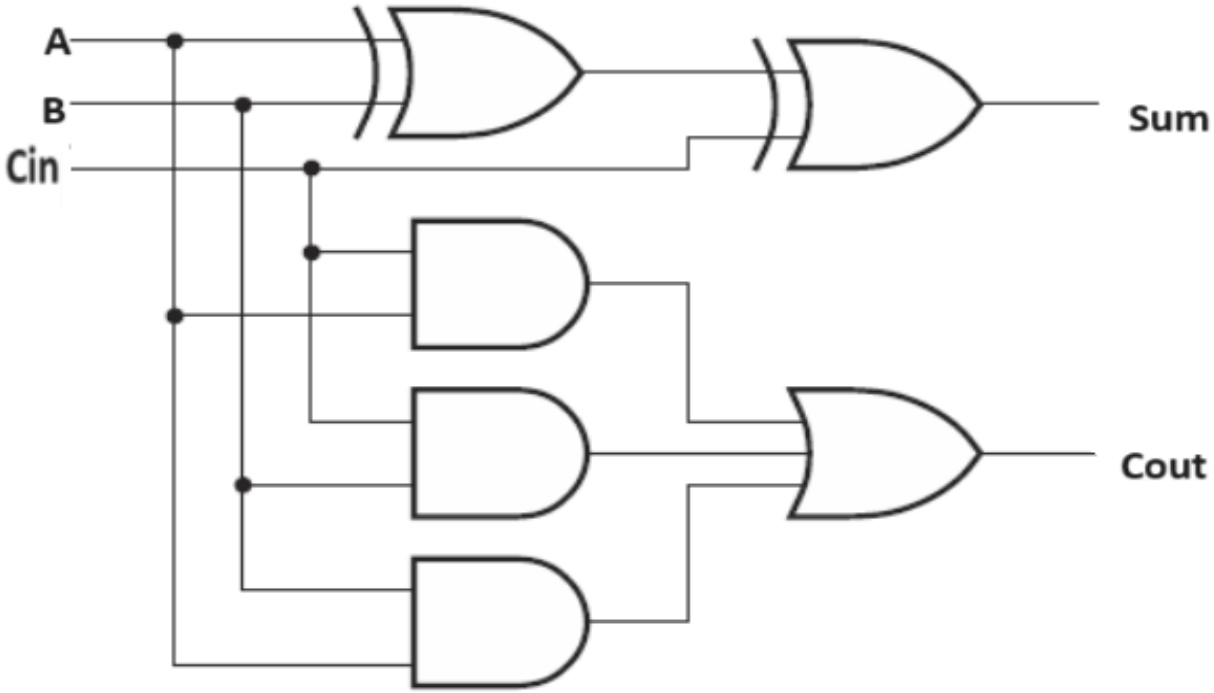
Design a logic circuit that can add three single binary digits (A, B, and a carry-in, Cin) and produce two outputs: the sum (S) of the three inputs and a carry-out (Cout) that represents any overflow from the addition.

Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Design of Combinational Circuit

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\overline{C_{in}} + A\bar{B}\overline{C_{in}} + ABC_{in}$$
$$= C_{in}(\bar{A}\bar{B} + AB) + \overline{C_{in}}(\bar{A}B + A\bar{B})$$
$$= C_{in}\overline{(A \oplus B)} + \overline{C_{in}}(A \oplus B)$$
$$= (A \oplus B \oplus C_{in})$$

$$C_{out} = AB + BC_{in} + C_{in}A$$



Design of Combinational Circuit

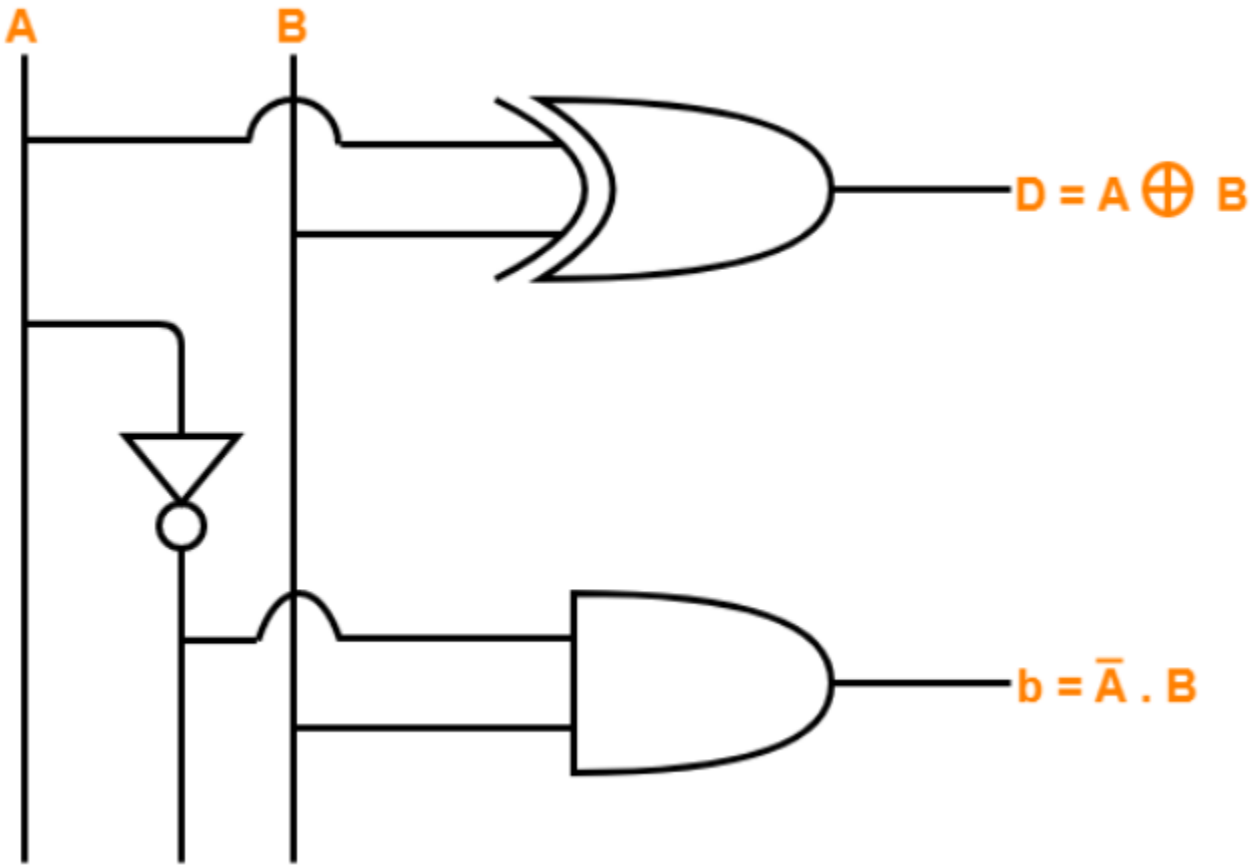
Design a digital logic circuit that performs binary subtraction of two single-bit binary numbers. It has two inputs, A and B, and two outputs, DIFFERENCE(D) and BORROW(b). The difference between the two input bits is the DIFFERENCE output, while the BORROW output indicates whether borrowing was necessary during the subtraction.

Inputs		Outputs	
A	B	D (Difference)	b (Borrow)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$D = A \oplus B$$

$$b = \overline{A}.B$$

Design of Combinational Circuit



Design of Combinational Circuit

Design a **combinational circuit** that performs subtraction of two bits, one is minuend and other is subtrahend, taking into account borrow of the previous adjacent lower minuend bit. This circuit **has three inputs and two outputs**. The three inputs A, B and Bin, denote the minuend, subtrahend, and previous borrow, respectively. The two outputs, D and Bout represent the difference and output borrow, respectively.

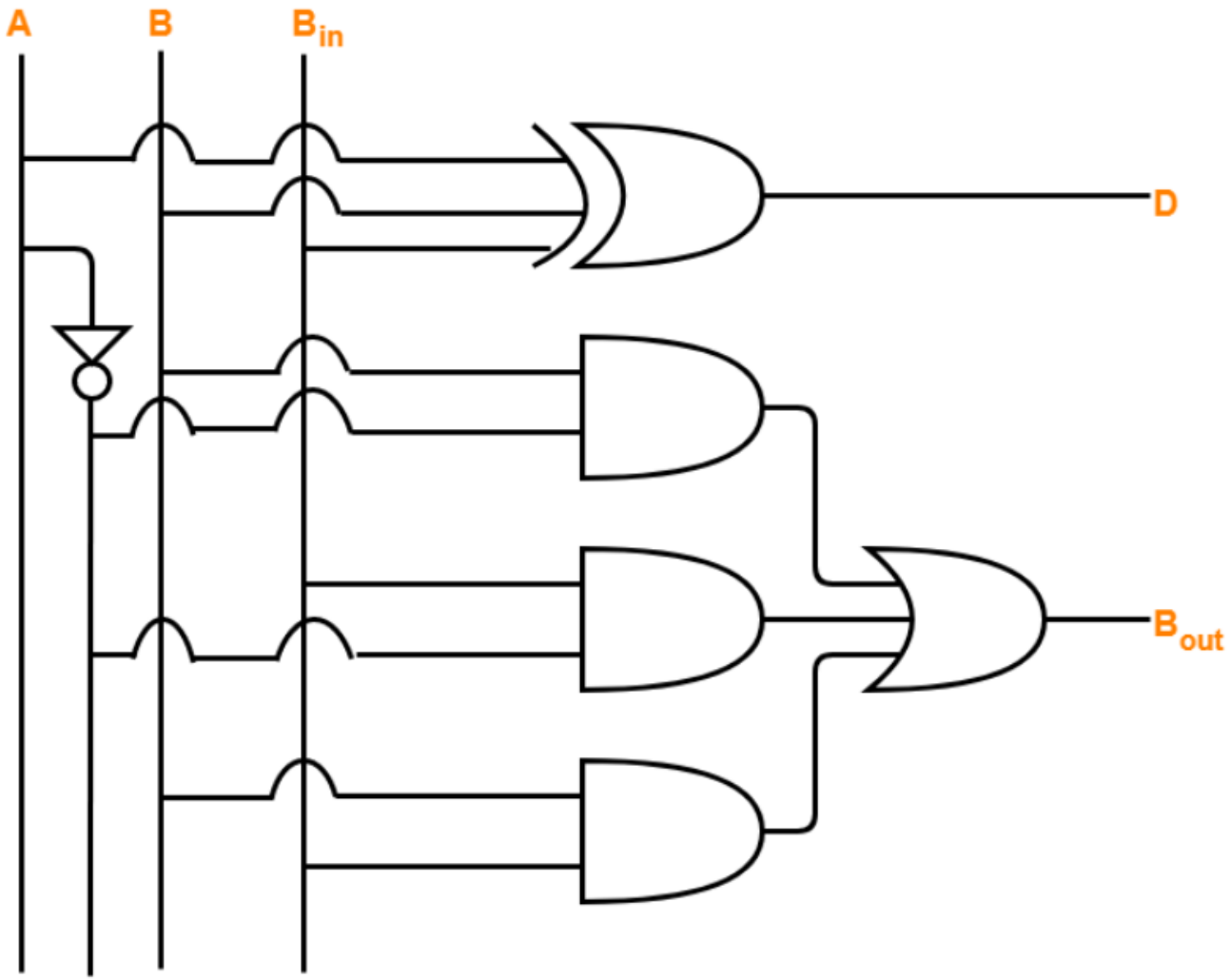
Design of Combinational Circuit

Inputs			Outputs	
A	B	B_{in}	B_{out} (Borrow)	D (Difference)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$D = A \oplus B \oplus B_{in}$$

$$B_{out} = \overline{A}\,B + (\overline{A} + B)\,B_{in}$$

Design of Combinational Circuit



Design of Combinational Circuit

Design a logic circuit whose output is HIGH only when a majority of the inputs A, B ,C and D are HIGH.

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

$$Y = \overline{A}BCD + A\overline{B}CD + AB\overline{C}D + ABC\overline{D} + ABCD$$

Y=(A·B·C)+(A·B·D)+(A·C·D)+(B·C·D)

Logic Diagram

- Also known as Switching equations
- A **literal** is a Boolean Variable or its complement (a, x', z)
- A **product term** is a literal or the logical product (AND) of multiple literals (ab, x'y, z'y')
- A **sum term** is a literal or the logical sum (OR) of multiple literals (a+b, a'+b, x'+y')
- A **sum of product (SOP)** is the logical OR of multiple product terms (ab+bc'+a'b')
- A **product of sum (POS)** is the logical AND of multiple sum terms[(a+b')(a'+b')(a'+b)]

Minterms

- A minterm is a special product of literals, in which each input variable appears exactly once.
- A function with n variables has 2^n minterms
- Each minterm is true for exactly one combination of inputs

Minterm	Is true when…	Shorthand
$x'y'z'$	x=0, y=0, z=0	m_0
$x'y'z$	x=0, y=0, z=1	m_1
$x'yz'$	x=0, y=1, z=0	m_2
$x'yz$	x=0, y=1, z=1	m_3
$xy'z'$	x=1, y=0, z=0	m_4
$xy'z$	x=1, y=0, z=1	m_5
xyz'	x=1, y=1, z=0	m_6
xyz	x=1, y=1, z=1	m_7

- Sum of minterms(canonical SOP) expression can be written from the truth table by picking out the rows of the table where the function output is 1.

x	y	z	f(x,y,z)	f'(x,y,z)
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	0	1

$f = x'y'z' + x'y'z + x'yz' + x'yz + xyz'$
$= m_0 + m_1 + m_2 + m_3 + m_6$
$= \Sigma m(0,1,2,3,6)$

$f' = xy'z' + xy'z + xyz$
$= m_4 + m_5 + m_7$
$= \Sigma m(4,5,7)$

f' contains all the minterms not in f

Maxterms

- A maxterm is a special sum of literals, in which each input variable appears exactly once.
- A function with n variables has 2^n maxterms
- Each maxterm is false for exactly one combination of inputs

Maxterm	Is *false* when…	Shorthand
$x + y + z$	x=0, y=0, z=0	M_0
$x + y + z'$	x=0, y=0, z=1	M_1
$x + y' + z$	x=0, y=1, z=0	M_2
$x + y' + z'$	x=0, y=1, z=1	M_3
$x' + y + z$	x=1, y=0, z=0	M_4
$x' + y + z'$	x=1, y=0, z=1	M_5
$x' + y' + z$	x=1, y=1, z=0	M_6
$x' + y' + z'$	x=1, y=1, z=1	M_7

- The product of maxterms(Canonical POS) expression can be obtained by picking out the rows of the table where the function output is 0.

x	y	z	f(x,y,z)	f'(x,y,z)
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	0	1

$$f = (x' + y + z)(x' + y + z')(x' + y' + z')$$
$$= M_4 M_5 M_7$$
$$= \prod M(4,5,7)$$

$$f' = (x + y + z)(x + y + z')(x + y' + z)(x + y' + z')(x' + y' + z)$$
$$= M_0 M_1 M_2 M_3 M_6$$
$$= \prod M(0,1,2,3,6)$$

f' contains all the maxterms not in f

Relation of Minterm to Maxterm

- Any minterm m_i is the *complement* of the corresponding maxterm M_i

Minterm	Shorthand	Maxterm	Shorthand
$x'y'z'$	m_0	$x + y + z$	M_0
$x'y'z$	m_1	$x + y + z'$	M_1
$x'yz'$	m_2	$x + y' + z$	M_2
$x'yz$	m_3	$x + y' + z$	M_3
$xy'z'$	m_4	$x' + y + z$	M_4
$xy'z$	m_5	$x' + y + z'$	M_5
xyz'	m_6	$x' + y' + z$	M_6
xyz	m_7	$x' + y' + z'$	M_7

SOP and POS forms

Boolean expressions can be written in the **sum-of-products** form (**SOP**) or in the **product-of-sums** form (**POS**).

An expression is in SOP form when two or more product terms are summed as in the following examples:

$\overline{A}\,\overline{B}\,\overline{C} + A\,B$ $\qquad$ $A\,B\,\overline{C} + \overline{C}\,\overline{D}$ $\qquad$ $C\,D + \overline{E}$

An expression is in POS form when two or more sum terms are multiplied as in the following examples:

$(A + B)(\overline{A} + C)$ $\qquad$ $(A + B + \overline{C})(B + D)$ $\qquad$ $(\overline{A} + B)C$

Canonical Forms

- A Canonical form of a switching equation is one that contains all of the available input variables.
  - **Canonical Sum of Products**: It is a complete set of Minterms that defines when an output is a logical 1
  - **Canonical Product of sums:** It is a complete set of maxterms that defines when an output is a logical 0
- Canonical expressions are not simplified and contains redundancies.

f = x'y' + x'y + xyz' is a normal expression
x'y'z' + x'y'z + x'yz' + x'yz + xyz' is the canonical form

Conversion to Canonical Form

- **SOP to canonical**:
  - Identify the missing variable in each AND term
  - AND the missing term and its complement with the original AND term
  - Expand the term by distribution property

Place the following equations into the proper canonical form:

a. $P = f(a, b, c) = ab' + ac' + bc$ (SOP)

1. First term, ab', is missing the variable c. So we AND $(c + c')$ with ab':

$$ab' = ab'(c + c') = ab'c + ab'c'$$

2. Second term, ac', is missing the variable b. So we AND $(b + b')$ with ac'.

$$ac' = ac'(b + b') = abc' + ab'c'$$

3. Third term is missing the variable a. So we AND $(a + a')$ with bc.

$$bc = bc(a + a') = abc + a'bc$$

4. The final canonical SOP form is

$$P = ab'c + ab'c' + abc' + ab'c' + abc + a'bc$$

Note that two terms, the second and the fourth, are identical. Only one is needed, because any variable or group of variables ORed with itself is redundant: $x + x = x$ (property of idempotency or sameness).

The final equation becomes

$$P = ab'c + ab'c' + abc' + abc + a'bc$$

b. $G = f(w, x, y, z) = w'x + yz'$ (SOP)

1. In this example the terms are missing two variables. The first term is missing variables y and z.

$$w'x = w'x(y + y')(z + z')$$

Expand in two stages:

$$(w'xy + w'xy')(z + z')$$

$$w'xyz + \quad xyz' + w'xy'z + w'xy'z'$$

2. The second term is also missing two variables.

$$yz'(w + w')(x + x')$$

Expand in two stages:

$$(yz'w + yz'w')(x + x')$$

$$yz'wx + yz'wx' + yz'w'x + yz'w'x'$$

Rearrange to reflect the original binary order and eliminate redundant terms.

$$G = f(w, x, y, z)$$
$$= wxyz' + wx'yz' + w'xyz + w'xyz' + w'xy'z + w'xy'z' + w'x'yz'$$

- **POS to canonical**:
  - Identify the missing variable in each OR term
  - OR the missing term and its complement with the original OR term
  - Expand the term by the distribution property

c. $T = f(a, b, c) = (a + b')(b' + c)$ (POS)

1. The variable c is missing from the first term.

$$a + b' + cc' = (a + b' + c)(a + b' + c')$$

2. The variable a is missing from the second term.

$$b' + c + aa' = (a + b' + c)(a' + b' + c)$$

3. The complete equation is

$$T = f(a, b, c) = (a + b' + c)(a + b' + c')(a' + b' + c)$$

d. $J = f(A, B, C, D) = (A + B' + C)(A' + D)$ (POS)

SOLUTION

1. $A + B' + C + DD' = (A + B' + C + D)(A + B' + C + D')$

2. $A' + D + CC' + BB'$

Distribute in two steps:

$(A' + D + C)(A' + D + C') + BB'$

$(A' + D + C + B)(A' + D + C + B')(A' + D + C' + B)$

$(A' + D + C' + B')$

3. The final canonical equation is

$J = f(A, B, C, D) = (A + B' + C + D)(A + B' + C + D')$

$(A' + B + C + D)(A' + B' + C + D)(A' + B + C' + D)(A' + B' + C' + D)$ ❖

Express the following SOP equations in a minterm list (shorthand decimal notation) form:

a. $H = f(A, B, C) = A'BC + A'B'C + ABC$

1. $A'BC = 011_2 = 3_{10}$.
2. $A'B'C = 001_2 = 1_{10}$.
3. $ABC = 111_2 = 7_{10}$.

Rearrange the decoded decimal values in numerical order:

$$H = f(A, B, C) = \sum(1, 3, 7)$$

b. $G = f(w, x, y, z) = wxyz' + wx'yz' + w'xyz' + w'x'yz'$

1. $wxyz' = 1110_2 = 14_{10}$.
2. $wx'yz' = 1010_2 = 10_{10}$.
3. $w'xyz' = 0110_2 = 6_{10}$.
4. $w'x'yz' = 0010_2 = 2_{10}$.

Rearrange the decimal equivalent minterms in numerical order:

$$G = f(w, x, y, z) = \sum(2, 6, 10, 14)$$

Numerical on Number System Conversions